

Executive Summary

This document presents the technical design, empirical data quality audit, physical solar modeling, ensemble learning, post-processing rules, and validation methodology for day-ahead hourly AC power generation forecasting at a 10 MW (12.5 MWp DC) utility-scale solar PV plant in Tumakuru, Karnataka, India (14.10° N, 77.28° E).

The forecasting engine converts day-ahead numerical weather forecasts (`forecast_cloud_cover`, `forecast_temp_c`, `forecast_wind_ms`, `forecast_humidity_pct`) into an hourly exported AC power schedule (`predicted_ac_power_kw`) for the 2-month test period from July 1, 2025 to August 31, 2025 (1,488 hourly predictions).

1. Problem Formulation & Operational Constraints

- **Input Horizon:** Day-ahead weather forecasts issued the previous afternoon.
- **Missing On-Site Inputs:** On-site SCADA sensor measurements (`measured_poa_wm2`, `measured_module_temp_c`, `status`, etc.) do not exist for the test period.
- **Physical Rating Limits:** Plant AC inverter capacity is **10,000 kW (10 MW)**; DC array capacity is **12,500 kWp** (DC/AC ratio 1.25).
- **Time Convention:** `timestamp` represents the **hour beginning** (HH:00:00 IST), while all row values represent the **mean over that clock hour** (HH:00 to HH+1:00).
- **Diurnal Hard Constraint:** Exported AC power at night (when solar elevation at hour midpoint $\leq 0^\circ$) is physically **0.0 kW**.

2. Empirical Data Quality Audit & Cleaning Rationale

`data/train.csv` is an unprocessed export from the plant historian. We performed a comprehensive audit to identify and resolve data quality anomalies:

A. Explicit Two-Format Timestamp Parser & Deduplication

- **Mixed Format Parser Bug Prevention:** Standard pandas `to_datetime(..., format='mixed', dayfirst=True)` erroneously swaps month and day for ISO dates (e.g., converting `2025-07-01 00:00:00` into `2025-01-07 00:00:00`).
- **Fix:** Implemented an explicit two-format timestamp parser (`parse_timestamp`):

```
python def parse_timestamp(s: pd.Series) -> pd.Series: iso = pd.to_datetime(s, format='%Y-%m-%d %H:%M:%S', errors='coerce') dmy = pd.to_datetime(s, format='%d-%m-%Y %H:%M', errors='coerce') return iso.fillna(dmy)
```

- **Deduplication:** Deduplicated `train.csv` across the true 36 duplicate timestamp pairs (72 duplicate rows total) by taking the median/mean of numeric columns and first valid SCADA status.

B. Sentinel Values (`-999.0` & `9999.0`) & Missing Data Imputation

- **Sentinel Replacement:** Sentinels (`-999.0` and `9999.0`) were present in `forecast_wind_ms` (14 rows), `measured_poa_wm2` (23 rows of `-999` and 29 rows of `9999`), `measured_module_temp_c` (39 rows), `measured_ambient_temp_c` (26 rows), and `ac_power_kw` (67 rows). Replaced with `NaN` **prior** to the deduplication aggregation step to prevent silent data pollution.
- **Test Set Missing Weather Data:** `data/test.csv` contained 19 distinct rows with missing/sentinel data (14 rows with `NaN` across forecast weather variables + 5 rows where `forecast_temp_c == -999.0`). Missing test weather features were imputed using time-based linear interpolation followed by forward/backward fill.
- **Train Set Target Cleaning:** Training rows with missing target values (`NaN` or `-999.0`) were dropped.

C. Sensor Glitches, Frozen Sensor Stretches & Outliers

- **Physical Capacity Spikes:** `train.csv` contained 29 rows $> 10,000 \text{ kW}$ and 13 severe corrupt sensor spikes reaching up to **26,905.99 kW** ($> 2.6 \times \text{AC rating}$). Training rows with `ac_power_kw > 10,500 kW` were filtered out.
- **Frozen Sensor Stretch Detection:** Detected a stuck SCADA sensor stretch of 48 consecutive identical readings of $2,187.44 \text{ kW}$ starting on 2024-09-18. Implemented a stuck-value detector (`filter_stuck_values`) flagging and removing runs of ≥ 4 consecutive identical non-zero values.
- **SCADA Non-Operational States:** Filtered out 263 rows where plant SCADA status was `STOP` or `PARTIAL` (curtailments, inverter trips, or maintenance) so the regression models learn true forecast capability rather than unannounced plant outages.

3. Hour-Midpoint Physics-Informed Feature Engineering

Hour Midpoint Solar Geometry

Because `timestamp` represents the **hour beginning** (HH:00) while values represent the **mean over the clock hour** (HH:00–HH+1:00), calculating solar position at HH:00:00 causes sunrise hours (e.g. 06:00 IST) to be erroneously flagged as night (elevation $< 0^\circ$ at 06:00:00, but elevation $> 0^\circ$ by 06:30:00). - **Fix:** All solar geometry and clear-sky equations are evaluated at the **hour midpoint**: $t_{\text{mid}} = t + 30 \text{ minutes}$. This preserves real morning generation ($\sim 87.4 \text{ kW}$ average at 06:00 IST) across 45/62 test sunrise hours. - **Dormant Bug Prevention:** `add_physics_and_weather_features` imports `parse_timestamp` directly from `src.clean` so fallback logic can never reintroduce date swapping.

Derived Physical Features

Using plant coordinates (14.10° N, 77.28° E, 680 m alt, 15° south tilt) via `pvlib` at t_{mid} : 1. **Astronomical Solar Position:** Solar zenith (θ_z), elevation (γ_s), azimuth (ϕ_s), and

cosine zenith ($\max(0, \cos \theta_z)$). 2. **Clear-Sky POA Irradiance:** Ineichen clear-sky Global Horizontal (GHI_{cs}), Direct Normal (DNI_{cs}), and Plane-Of-Array (POA_{cs}) irradiance on 15° south tilt. 3. **Kasten-Czeplak Cloud Attenuation:** $POA_{est} = POA_{cs} \cdot (1 - 0.75 \cdot C^{3.5})$ for cloud cover fraction C . 4. **Faiman Cell Thermal Model:** $T_{module} = T_{ambient} + \frac{POA_{est}}{25} + 1.2 \cdot W_{wind}$. 5. **Theoretical Generation Output:** $P_{dc_est} = P_{dc_cap} \cdot \left(\frac{POA_{est}}{1000}\right) \cdot \left[1 - 0.0035 \cdot (T_{module} - 25)\right]$ $P_{ac_est} = \min(10000.0, P_{dc_est} \cdot 0.97)$ 6. **Cyclical Temporal Encodings:** Trigonometric sine/cosine encodings of clock hour, month, day of year, and solar noon offset.

4. Modeling Architecture & Dynamic Meta-Learner

We trained a 5-fold cross-validated GBDT ensemble blending: 1. **LightGBM Regressor** (Leaf-wise gradient tree growth) 2. **CatBoost Regressor** (Symmetric oblivious trees) 3. **XGBoost Regressor** (Histogram depth-wise trees)

Dynamic Meta-Learner Weight Fitting

Meta-blending weights are learned dynamically via Non-Negative Least Squares (Ridge regression constrained ≥ 0) on Out-of-Fold (OOF) predictions: $\hat{y}_{ensemble} = 0.180 \cdot \hat{y}_{LightGBM} + 0.556 \cdot \hat{y}_{CatBoost} + 0.264 \cdot \hat{y}_{XGBoost}$

5. Physical Post-Processing Rules

- 1. **Midpoint Nighttime Cutoff:** If $\gamma_s \leq 0^\circ$ at t_{mid} or $POA_{cs} \leq 1.0 \text{ W/m}^2$ implies $\hat{y} = 0.0 \text{ kW}$.
- 2. **AC Inverter Capacity Cap:** $\hat{y} = \min(10000.0 \text{ kW}, \hat{y})$.
- 3. **Non-Negativity Constraint:** $\hat{y} = \max(0.0 \text{ kW}, \hat{y})$.

6. Model Benchmarking & Monsoon Seasonal Validation

A. Validation Methodology

We evaluate models across two validation schemes: 1. **5-Fold Cross Validation:** Computed dynamically on the full historical record (12,304 cleaned rows). 2. **Seasonal Monsoon Holdout Validation:** Training on all data **except** July 1, 2024 – August 31, 2024, and testing strictly on July 1, 2024 – August 31, 2024. This simulates the exact seasonal regime shift of predicting the July–August 2025 monsoon test set.

B. Dynamic Benchmark Performance Comparison

All metrics below are computed dynamically at runtime by `preprocessing/baselines.py` from a single execution run:

Model / Strategy	5-Fold OOF RMSE (kW)	5-Fold OOF MAE (kW)	Jul–Aug 2024 Monsoon Holdout RMSE (kW)	Jul–Aug 2024 Monsoon Holdout MAE (kW)
Persistence Baseline (\$t - 24\text{h}\$)	1,165.61	446.05	1,177.88	442.59
Physics-Only Analytical (\$P_{\text{ac_est}}\$)	998.45	409.95	2,596.42	1,220.01
Raw Weather LightGBM (No Physics)	561.27	186.78	2,399.54	1,138.82
Final Physics GBDT Ensemble	258.45	92.94	2,402.95	1,137.34

7. Model Weaknesses & Honest Risk Assessment

- Monsoon Regime Generalization Gap:** While 5-fold cross validation yields an OOF RMSE of **258.45 kW** ($R^2 = 0.9942$), the seasonal holdout validation on July–August 2024 reveals a realistic monsoon error of **2,402.95 kW**. This error gap stems from:
- Single Historical Monsoon in Training Set:** The training dataset spans Jan 2024 – Jun 2025, containing only *one* prior monsoon season (Jul–Aug 2024).
- Day-Ahead Cloud Forecast Uncertainty:** Monsoon rainstorms produce rapid, localized cloud fluctuations that day-ahead NWP models often mispredict.
- Monsoonal Aerosol & Diffuse Scattering:** High relative humidity (>85%) and heavy aerosol loading alter atmospheric diffuse scattering ratios beyond standard clear-sky models.
- Unannounced SCADA Outages:** SCADA status is unavailable for the test set. Unannounced grid curtailments or inverter trips during August cannot be predicted from weather forecasts alone.

8. Future Improvements

- Probabilistic / Quantile Loss Ensembling:** Train Quantile GBDT models (Pinball loss) to output P10, P50, and P90 confidence intervals for grid scheduling.
- Multi-NWP Satellite Stacking:** Blend ECMWF, GFS, and NCUM numerical weather forecasts to mitigate single-provider cloud cover forecast error.
- Deep Learning Temporal Architectures:** Train N-BEATS or Temporal Fusion Transformers (TFT) to model multi-hour day-ahead horizon dependencies.